

User Manual

Project Title: Arithmetic Expression Evaluator in C++
Course: EECS 348 — Software Engineering,
Deliverable: User Manual
Version: 1.0
Date: May 10, 2026
Repository: <https://github.com/WillScoot-crtl/AEEDevelopment>

Team Members

Name	Role
Ritwik Goyal	Product Manager
Mahathi Sai Palukuri	DevOps Engineer
Ayesha Akbar	Quality Assurance

Revision History

Date	Version	Description	Author
May 10, 2026	1.0	Initial user manual document	Team AEEDevelopment

1. Introduction

1.1 Purpose

This user manual explains how to install, run, and use the Arithmetic Expression Evaluator in C++ project. It is written for users of different experience levels and is intended to help them understand the software’s purpose, supported features, workflow, and common issues.

1.2 Software Overview

The Arithmetic Expression Evaluator is a C++ program that reads arithmetic expressions entered by the user, parses them, evaluates them, and returns either the correct result or a clear error message. The project is based on the requirements and architecture defined in the earlier deliverables.

1.3 Main Features

The current version of the software supports the following features:

- Integer arithmetic expressions.
- Binary operators: $+$, $-$, $*$, $/$, $\%$, and $**$.
- Unary operators: $+$ and $-$.
- Parentheses for grouping.
- Operator precedence and associativity.
- Error handling for invalid input.
- Error handling for divide-by-zero and modulo-by-zero cases.
- A command-line interface for entering expressions.

1.4 Intended Users

This manual is intended for instructors, graders, students testing the software, team members continuing development, and users who want a simple arithmetic expression evaluator.

1.5 Installation and Run Overview

The current implementation is organized as a C++ project with source files, header files, tests, a README, and a CMake build file. The software is currently used through a command-line interface rather than a full GUI.

2. Getting Started

2.1 Prerequisites

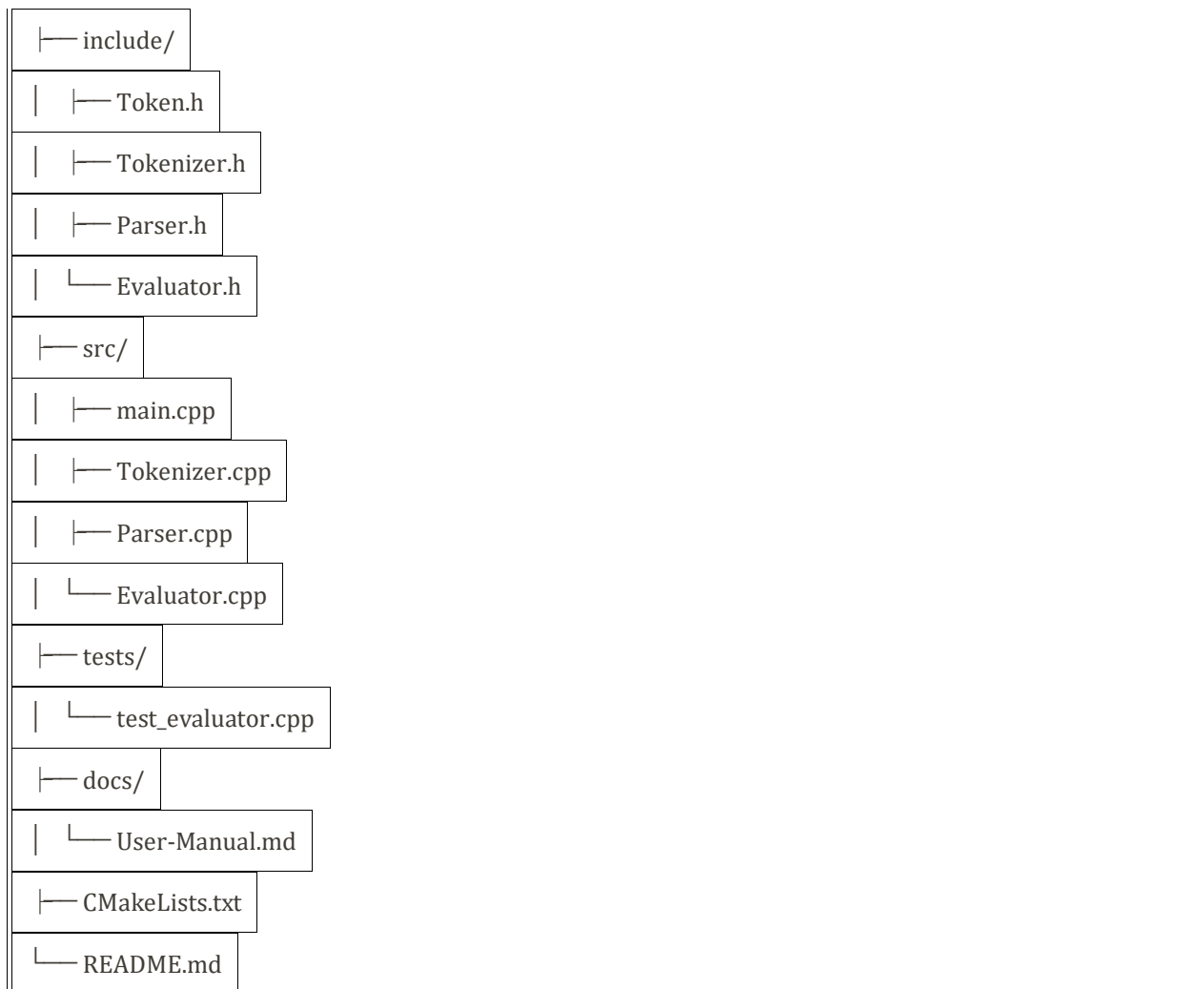
To use the software, the user should have:

- A C++ development environment.
- Access to the project repository.
- The source files and build configuration.
- A terminal or command-line window.

2.2 Repository Structure

The project is organized into the following major folders and files.

```
text
AEEDevelopment/
```



2.3 Building the Software

Use the following steps to build the program.

1. Open a terminal.
2. Navigate to the project folder.
3. Create a build directory.
4. Run the build commands.

Example:

```
mkdir build
cd build
cmake ..
cmake --build .
```

2.4 Running the Software

After building the project, run the executable from the build directory.

```
/aeedevelopment
```

When the program starts, it shows a prompt where the user can enter an arithmetic expression.

2.5 Exiting the Software

To exit the program, type:

```
quit
```

or

```
exit
```

3. Using the Software

3.1 Basic Workflow

The normal user workflow is:

1. Start the program.
2. Type an arithmetic expression.
3. Press Enter.
4. Read the result or error message.
5. Continue with another expression or exit the program.

3.2 Entering Expressions

Users can enter arithmetic expressions using integers, supported operators, and parentheses.

Examples of valid input include:

- ```
3 + 4
```
- ```
8 - (5 - 2)
```
- ```
10 * 2 / 5
```
- ```
2 ** 3
```
- ```
2 ** 3 ** 2
```
- ```
4 * (3 + 2) % 7 - 1
```

- `-(+1) + (+2)`

3.3 Supported Operators

Operator	Meaning	Example
<code>+</code>	Addition	<code>3 + 2</code>
<code>-</code>	Subtraction	<code>8 - 5</code>
<code>*</code>	Multiplication	<code>4 * 6</code>
<code>/</code>	Division	<code>20 / 5</code>
<code>%</code>	Modulo	<code>17 % 5</code>
<code>**</code>	Exponentiation	<code>2 ** 3</code>
unary <code>+</code>	Positive value	<code>+4</code>
unary <code>-</code>	Negative value	<code>-4</code>

3.4 Understanding Results

If the expression is valid, the program prints a result.

Example:

```
> 3 + 4
```

```
Result: 7
```

If the expression is invalid, the program prints an error.

Example:

```
> 4 / 0
```

```
Error: Division by zero
```

3.5 Precedence and Associativity

The software follows arithmetic precedence and associativity rules from the project requirements.

- Parentheses are evaluated first.
- Exponentiation uses right associativity.
- Multiplication, division, and modulo are evaluated before addition and subtraction.
- Addition, subtraction, multiplication, division, and modulo use left associativity.

Examples:

- `3 + 4 * 2` produces `11`.
- `(3 + 4) * 2` produces `14`.
- `2 ** 3 ** 2` produces `512`.
- `10 - 3 - 2` produces `5`.

4. Advanced Features

4.1 Current Advanced Features

The current implementation includes several advanced evaluation features even though the interface is simple.

These include:

- Nested parentheses.
- Unary operators.
- Exponentiation.
- Modulo support.
- Structured error handling.

4.2 Planned or Future Features

Earlier project documents discussed more advanced interface and visualization features that may be added later. These may include a graphical user interface, parse tree visualization, token display, and step-by-step evaluation display.

These features are part of the project direction but are not part of the current command-line usage workflow.

4.3 Not Currently Supported

The current version does not support custom variables, user-defined functions, saving expressions to files, loading expressions from files, floating-point evaluation, or negative exponent evaluation in integer-only mode.

5. Troubleshooting

5.1 Common Problems and Solutions

Problem	Possible Cause	Solution
Program does not start	Build not completed or executable not found	Rebuild the project and verify the executable name and build location.

Expression gives an error	Expression may be malformed	Check parentheses, operator placement, and valid syntax.
Unsupported character error	Expression contains an unsupported symbol	Use only integers, supported operators, and parentheses.
Division by zero error	Divisor is zero	Change the expression so the divisor is not zero.
Modulo by zero error	Modulo divisor is zero	Change the expression so the modulo divisor is not zero.
Missing closing parenthesis	Opening parenthesis has no matching closing parenthesis	Review and balance all parentheses.
Unexpected result	Precedence or associativity may be misunderstood	Add parentheses to make the intended grouping explicit.
Negative exponent rejected	Current version is integer-based	Use non-negative exponents only.
Empty input causes an error	No expression was entered	Enter a complete arithmetic expression before pressing Enter.

5.2 Example Error Cases

Invalid character

> 7 & 3

Error: Unsupported character: &

Missing parenthesis

> 2 * (4 + 3 - 1

Error: Missing closing parenthesis

Malformed input

> * 5 + 2

Error: Expected number or parenthesized expression

6. Examples

6.1 Simple Expressions

$$3 + 4 \rightarrow 7$$

$$10 - 6 \rightarrow 4$$

$$5 * 7 \rightarrow 35$$

$$20 / 5 \rightarrow 4$$

$$17 \% 5 \rightarrow 2$$

6.2 Parentheses and Precedence

$$3 + 4 * 2 \rightarrow 11$$

$$(3 + 4) * 2 \rightarrow 14$$

$$((2 + 3) * (4 - 1)) \rightarrow 15$$

6.3 Unary Operators

$$-(2 * (-3)) \rightarrow 6$$

$$- (+1) + (+2) \rightarrow 1$$

6.4 Exponentiation

$$2 ** 3 \rightarrow 8$$

$$2 ** 3 ** 2 \rightarrow 512$$

6.5 Error Examples

$$4 / 0 \rightarrow \text{division by zero error.}$$

$$10 \% 0 \rightarrow \text{modulo by zero error.}$$

$$7 \& 3 \rightarrow \text{unsupported character error.}$$

$$5 (2 + 3) \rightarrow \text{malformed expression error.}$$

7. FAQ

Q1. What kinds of numbers can I enter?

You can enter integer values.

Q2. Does the software support parentheses?

Yes. Parentheses are fully supported and can be nested.

Q3. Does the software support operator precedence?

Yes. The evaluator follows the precedence and associativity rules described in the earlier requirements and implementation materials.

Q4. Can I enter spaces in an expression?

Yes. Extra spaces are allowed in valid expressions.

Q5. Can I save expressions or define variables?

No. Those features are not included in the current implementation.

Q6. Why does `2 ** -3` fail?

The current implementation is integer-based and does not support negative exponents.

Q7. What should I do if I get an unexpected result?

Check whether precedence or associativity affected the result, and use parentheses if you want to force a different grouping.

8. Glossary

Term	Meaning
Arithmetic expression	A mathematical expression made from numbers, operators, and parentheses.
Binary operator	An operator that works on two values, such as <code>+</code> or <code>*</code> .
Unary operator	An operator that works on one value, such as unary <code>-</code> .
Token	A recognized unit of input, such as a number or operator.
Parser	The part of the program that builds the internal expression structure.
Evaluator	The part of the program that computes the result.
Precedence	The rule that determines which operations are performed first.
Associativity	The rule that determines the order of evaluation when operators have the same precedence.
Command-line interface	A text-based interface where the user types commands or expressions.

9. Referenced Project Documents

This user manual is based on the earlier project artifacts and implementation materials, including the Project Description, Project Plan, Software Requirements Specification, Software Architecture, Programming Guidelines, Configuration Management Plan, Risk List, Test Cases, Implementation Deliverable, and README.